

Sub-model Parallelism: A Scale-out Deployment Method for Large Multi-modal DNNs

Tianhao Huang
Shanghai Jiao Tong University
Shanghai, China
hth_2003@sjtu.edu.cn

Lingyu Sun
Shanghai Jiao Tong University
Shanghai, China
sunlingyu@sjtu.edu.cn

Xiaofeng Hou
Shanghai Jiao Tong University
Shanghai, China
hou-xf@cs.sjtu.edu.cn

Xiaozhi Zhu
Shanghai Jiao Tong University
Shanghai, China
zhuxiaozhi@sjtu.edu.cn

Xinfeng Xia
Shanghai Jiao Tong University
Shanghai, China
xf6767@sjtu.edu.cn

Yutong Wang
Shanghai Jiao Tong University
Shanghai, China
yuki666@sjtu.edu.cn

Mingxi Chen
Shanghai Jiao Tong University
Shanghai, China
zeisesalt@sjtu.edu.cn

Chao Li
Shanghai Jiao Tong University
Shanghai, China
lichao@cs.sjtu.edu.cn

Abstract—We have witnessed an increasing usage of multi-modal DNNs with multi-task heads on edge computing scenarios. These networks typically process inputs of different modalities first, then extract features for unified fusion, and finally input the fused features into multi-task heads. Such networks are often used to determine pose and navigate movement direction via multi-modal data obtained from diverse sensory equipment, therefore necessitating low inference latency. An edge device cluster with high-speed interconnection can be employed to support such DNN workload for scaled-out performance. For accelerating model inference on edge devices, previous researchers have proposed methods including model pruning, quantization, etc. However, these methods failed to take advantage of the structural features of multi-modal DNNs with multi-task heads and may impair the model's prediction accuracy.

Based on the intrinsic structure of multi-modal DNNs with multi-task heads, we propose *Sub-model Parallelism* to achieve scalable execution speedup. Sub-model Parallelism is a scale-out deployment method that first assigns preprocessing tasks of different modalities to different edge devices, then delivers them to a device for modality feature fusion, and finally distributes the fused features to other devices responsible for different task head computations. We run experiments on *BEVFusion* network and achieve an approximately 30% reduction in latency using two *Jetson Orin* devices connected by *Remote Direct Memory Access* (RDMA). Furthermore, we conduct a series of simulation experiments to cover scale-out scenarios and also achieve a good level of latency reduction. We hope that our proposed method can provide valuable experience for the optimized scale-out deployment of large multi-modal DNNs with multi-task heads on multiple edge devices.

Index Terms—Sub-model Parallelism, Multi-modal DNN, Latency Reduction, Scale-out Deployment, Edge Devices

I. INTRODUCTION

Multi-modal DNNs (Deep Neural Networks) have become an increasingly prominent research focus in recent years [1]–

[3]. These networks typically involve a diverse array of modalities as input, including but not limited to text, vision, audio, and sensor data. Such models can fuse multi-sensory inputs with multiple *encoders* to perform multiple tasks simultaneously by attaching multiple *heads*. With transformer models gaining popularity for their outstanding performance in natural language processing and image processing, the computation requirements of some recent multi-modal DNNs with multi-task heads used in edge applications have become very large [4]. Thus, latency requirement is a critical challenge due to the limitation of computational resources possessed by edge devices. Researchers have launched many methods to accelerate the inference speed of the large DNNs on edge such as model partitioning, parallel execution, right-sizing, model pruning and so on [5]–[10]. These methods can be summarized into two aspects: *model compression* and *computing capability enhancement*.

Model compression methods perform tradeoff between model execution latency and prediction accuracy. For instance, model quantization method convert weight parameters of a DNN model from floating points to integers for computation reduction. However, it does not scale-up with more powerful device or scale-out with more devices. While model compression is one solution, enhancing the device's computing capability would be preferred to deploy the larger model without prediction accuracy loss. To achieve this, we can explore both scale-up and scale-out approaches. The scale-up approach has its limitations. Designing circuits for larger chips brings higher technological complexity, and the hardware performance of individual devices may face physical constraints. For instance, an overly concentrated computational workload may lead to challenges in heat dissipation. Moreover, in scenarios requiring

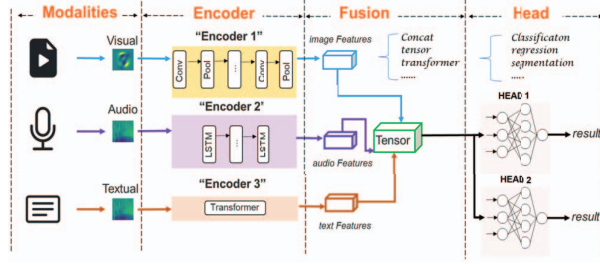


Fig. 1. Structure of multi-modal DNNs with multi-task heads.

high reliability, a single device failure in the scale-up scenario could lead to an unavailable system whereas scale-out with multiple devices can tolerate partial device failures to some extent. Additionally, the scale-out approach is more flexible, allowing for easier expansion and upgrades in the later stages. Therefore, our primary focus is on the scale-out approach. In the context of scale-out, we need to address the issue of parallelization across multiple devices. While there has been existing works in this area, current parallelization methods still fall short of meeting the particularly high latency requirements in some scenarios.

We propose **Sub-model Parallelism** as a novel solution. Sub-model Parallelism involves decomposing the encoder and head sub-models of the network into multiple branches when given two or more edge devices, each dedicated to processing specific sub-model, thereby achieving more efficient computation. The speedup of this approach can be proportional to the amount of edge devices used for large models with enough encoders and heads. We hope that our proposed method can provide some valuable experience for the scale-out deployment of large multi-modal DNNs with multiple task heads on multiple edge devices.

II. BACKGROUND AND MOTIVATION

A. Multi-modal DNNs with Multi-task Heads

Many studies have shown that multi-modal DNNs can significantly improve prediction accuracy compared to traditional uni-modal DNNs [11]–[14]. Multi-modal DNN utilizes multiple input modalities (such as images, text, sound) for training and inference. These models are capable of simultaneously processing information from multiple sensors or data source to improve prediction accuracy. This ability to comprehensively process data from multiple modalities makes multi-modal DNNs widely applicable in fields such as natural language processing, computer vision, and intelligent systems. Multi-modal DNNs typically consist of three sub-models: encoder, fusion, and head. Each encoder sub-model extracts feature from its corresponding modality input. A fusion sub-model integrates features from different modalities. Multiple head sub-models are attached thereafter to process the integrated feature and give prediction results for different tasks.

We demonstrate an example of multi-modal DNN with multi-task heads in Fig. 1. An important characteristic of

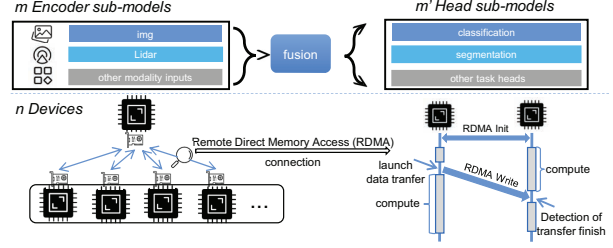


Fig. 2. Software abstraction and hardware architecture overview.

multi-modal DNNs is their ability to handle different types of input data, such as images, text, and sound, and extract rich information to support more accurate decision-making and predictions. In the encoder stage, independent neural networks are used to transform different input modalities into high-level representations (features). Convolution based sub-models are commonly used for image processing, while transformer based sub-models are specialized in text and audio processing. In the fusion stage, the feature representations from multiple encoders are integrated for joint learning. In the head stage, the network components specific to different tasks generate the final predictions or outputs. Some representative tasks include object detection, scene segmentation, emotion recognition, etc.

In edge application scenarios such as autonomous driving and intelligence robots, multi-modal DNNs are widely used because of the multi-modal characteristics of perceptual data [15], [16]. Such DNNs need to process camera images, radar point-cloud, lidar point-cloud and various sensor inputs to perform tasks including object detection, scene segmentation, localization, etc. The prediction results of these tasks would then be used to guide the actions of cars or robots. Therefore, edge inference of multi-modal networks is an important and worth discussing issue. There is a growing need for efficient and scalable approaches to handle low-latency inference execution of large-parameter multi-modal DNNs with multi-task heads on edge.

B. Ultra-low Latency Requirement Challenges

For edge inference application scenarios such as autonomous driving and intelligent robots, there are often higher requirements for the inference speed of the network [17]–[19]. Networks used to determine posture and control actions often require completion within tens of milliseconds. Faster completion speed under the premise of maintaining basic accuracy requirements means higher safety. Meanwhile, the scale of multi-modal DNNs with multi-task heads on edge is continuously expanding. This is resulted from the increasing availability of sensor data from different modalities and the growing complexity of models handling these modalities. This expansion signifies the need for scalable usage of computational power at the edge.

For example, *BEVFusion* network is a multi-sensor fusion application proposed in autonomous driving system unifying multi-modal features in the shared bird's-eye view (BEV)

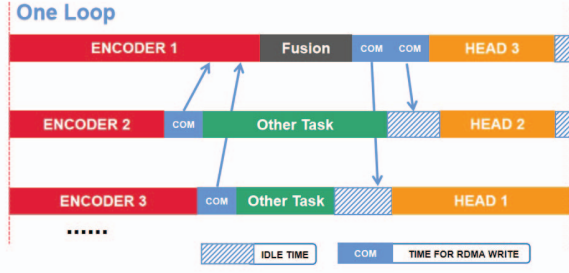


Fig. 3. Sub-model Parallelism on three encoder and head sub-models in a single loop as an example.

representation space and performing both object detection and segmentation tasks [20]. It is used to determine the position of the car and the distance to the surrounding environment. Therefore, it needs to be accomplished with ultra-low latency requirement. However, this large network also demands high computational power. Therefore, it is useful to find a way to obtain scalable latency reduction when scaling the underlying hardware platform to provide more computation capability.

In the context of providing scalable performance at edge, it is not always possible to satisfy the ultra-low latency requirement by scale-up using a single high-end device due to power, space, cost and fault tolerance considerations. Therefore, our research focuses on exploring methods for running multi-modal DNNs with multi-task heads with low latency on edge devices through scale-out techniques.

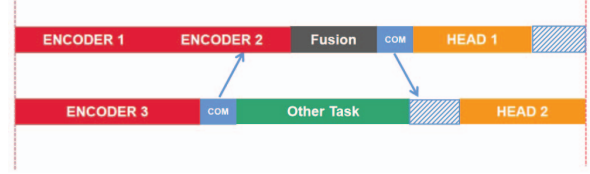
III. PROPOSED METHOD

A. Architecture Overview

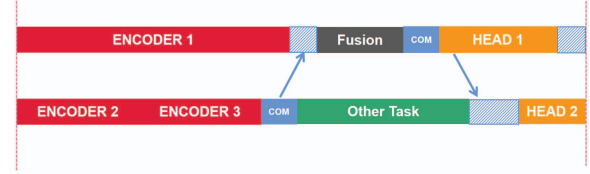
In this work we consider deploying large multi-modal DNNs with multi-task heads on scale-out architectures. We aim at achieving scalable performance on an edge device cluster with high-speed interconnection. Our target software workload and hardware platform is shown in Fig. 2

Specifically, the target hardware platform is a scalable edge cluster composed of multiple edge devices equipped with GPU for DNN execution. A typical example of such edge device is the popular *Jetson Series* manufactured by NVIDIA, and we pick *Jetson Orin*, the latest and most powerful one, to conduct all experiments in the remainder of this paper.

All edge devices in the cluster are connected to a master node using network cards with Remote Direct Memory Access (RDMA) support. RDMA allows data to be transferred directly between the memory of two connected computers without involving the processors or operating systems after the initialization work was completed. This enables high-speed, low-latency communication over a network, making it valuable for applications requiring high performance, such as data center networking and high-performance computing. The advantage of RDMA communication is that the CPU or GPU only needs a short time to hand over the data to the RDMA network card, and then it can be free from managing the transmission. Moreover, once the basic framework of RDMA



(a) Case 1: Encoder 3 is shorter than Encoder 1 plus Encoder 2.



(b) Case 2: Encoder 3 is longer than Encoder 1 plus Encoder 2.

Fig. 4. Applying Sub-model Parallelism when the number of encoders is larger than the number of device.

communication is established between both sides, one side can write data without the other side's knowledge, and the other side only needs to detect it when necessary. This significantly reduces the time occupied by communication and provides excellent support for parallelism between data transmission and computation.

The main challenge to be solved in this paper is to satisfy ultra-low latency requirement when deploying large multi-modal DNNs with multi-task heads on such edge clusters. A proper deployment approach is required to properly scale-out the target DNN workload.

B. Sub-model Parallelism

Based on the structure of multi-modal DNN with multi-task heads, we propose *Sub-model Parallelism*, an approach to reduce its execution latency in a scalable manner to satisfy ultra low-latency requirement at edge. We assume that the execution request of a target DNN arrives at the edge cluster periodically. Here we first consider a single execution in one period, and subsequent runs will adopt the same strategy. As shown in Fig. 3, we arrange each encoder sub-models on each devices and utilize the different running time of different encoder sub-models for RDMA communication. The device which undertakes relatively light-weight encoder sub-model will write the encoded features to the one undertaking the heaviest encoder sub-model. Thus, the communication time is overlapped and can help reduce whole latency. When all the encoder sub-models have finished execution, their extracted features will be written to a master device responsible for fusion, which is the one executing the longest encoder sub-model. After the fusion task is done, the fused feature will be written to other devices by the master. Heavier task head sub-model is assigned to the device who receives the fused feature earlier. The most light-weight task head is assigned to master device. ***This process achieves more efficient computation and communication based on different sub-model execution time.***

We admit that there exists idle periods of non-master devices during the execution of the fusion stage by the master device. These idle periods can be filled with other DNN execution requests arrived at the edge cluster during the execution of our target DNN by a traditional cluster scheduler. This helps to better utilize of the computing resources of the edge cluster.

We then formally describe the theoretical speedup when Sub-model parallelism is applied. Assuming we have n devices for m encoder sub-models and m' heads, if we take one device to run all the tasks and the others idle for backup, the whole latency is shown in Equation (1). Sub-model Parallelism could help reduce it to Equation (2). Because when using RDMA, the actual time spent on communication by the sender is very short, for most of the transmission time is completed independently by the RDMA network card. Therefore, we believe that the main running time is still determined by the maximum running time of encoders and heads.

$$L_{base} = \sum_{i=1}^m T_{en_i} + T_{fu} + \sum_{i=1}^{m'} T_{h_i} \quad (1)$$

$$L_{SP} = \max_{i \in m} (T_{en_i}) + T_{fu} + \max_{i \in m'} (T_{h_i}) + T_{com} \quad (2)$$

Through the above formula, we can see that *the degree to which we can reduce latency is closely related to the number of encoder and head sub-models*. The more sub-models a DNN has, the more devices we can use. Basically n encoder and head sub-models can correspond to nearly n times acceleration ratio (the time for fusion is generally short). We can also find the key for time saving is decided by the comparison of communication time and the execution time of the longest encoder and head sub-models. With high-speed interconnection, the communication time can be ignored. When the communication time is much less than block running time, our method can help reduce the latency for whole network significantly.

Fig. 3 has demonstrated a special case where $m = m' = n$. Sometimes the number of devices is fixed but the number of sub-models is larger, and sometimes the number of heads doesn't necessarily correspond exactly to the number of encoders. In short, $m = m' = n$ does not always hold. But we can also take advantage of Sub-model Parallelism in certain circumstances. For example, if $m > n$, some of the encoders can be placed on one same device, and we can still enjoy the benefit of latency reduction of Sub-model Parallelism. Two examples are shown in Fig. 4. The final latency contribution of encoders is then decided by the device whose encoder combination has the longest latency. Since the number of encoder sub-models in real networks would not be infeasibly large, a simple way to obtain an allocation scheme that minimizes the difference in execution times for encoding on two devices can be achieved through exhaustive enumeration. It is worth noting that we do not cover all cases, but aim to demonstrate through representative examples to show that our approach is scalable.

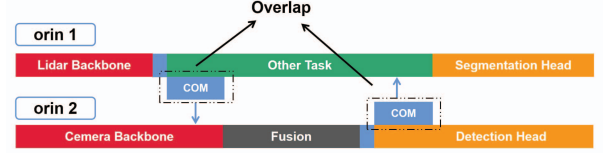


Fig. 5. A demonstration of applying Sub-model Parallelism to BEVFusion model on two Jetson Orins connected by RDMA.

IV. EXPERIMENTS AND EVALUATIONS

A. Evaluation Methodology

We run experiments on a platform containing two Nvidia Jetson Orins. We choose the BEVFusion model as our workload for demonstration. The reasons for choosing BEVFusion are as follows:

- BEVFusion is a novel DNN in the field of automatic driving, which is in line with the edge application scenarios.
- BEVFusion is a multi-modal DNN, which fits the application scenarios of Sub-model Parallelism.
- BEVFusion has an easy-to-use implementation for deployment, where the intermediate result can be easily acquired and transferred between two devices.

In short, the BEVFusion network consists of a lidar backbone and a camera backbone, which are used to process image data and lidar point-cloud data respectively. After processing, there is a fusion block to fuse the data, followed by two task heads for object detection and semantic segmentation. This is a standard multi-modal DNN with multi-task heads including two encoders and two heads. Its execution pattern after applying Sub-model Parallelism is applied in Fig. 5.

We use the *perftest* (for RDMA communication) and *Lidar_AI_Solution* (for deployment implementation of the BEVFusion model) *Github repository* [32], [33] as the foundation of the code. Perftest is a repository for measuring RDMA communication performance. We modify the code for testing *RDMA Write* in it to implement RDMA communication. We transplant the code from Perftest into Nvidia BEVFusion, so that after the execution of sub-models of BEVFusion model, an RDMA communication could be done. The initialization process of RDMA communication setup is done before the edge cluster accepts any model execution requests. When running the BEVFusion network, *RDMA Write* operation can be issued without initialization overhead. Furthermore, during the process of sub-model execution, we start a separate thread to perform RDMA communication. This enables transferring feature data simultaneously during sub-model execution.

B. Evaluation Results

We compare the running latency between Sub-model Parallelism and Non-Parallelism under four conditions. We pick four multi-modal DNN workloads as follows: (a) original BEVFusion; (b) A variant of BEVFusion with $m = m' = n = 3$; (c) $m = 3, m' = n = 2$, and two encoders on the same device take longer; (d) $m = 3, m' = n = 2$, but one

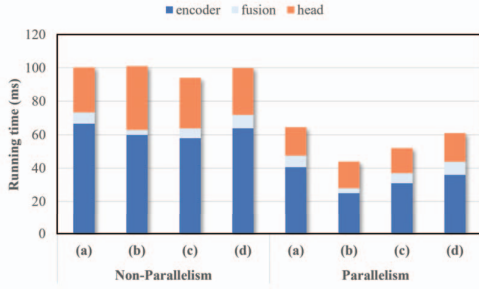


Fig. 6. Comparison of total running latency between Non-Parallelism and applying Sub-model Parallelism. (a): *original BEVFusion*, b,c,d: *extended variants of BEVFusion*, corresponding to Fig. 3, Fig. 4, and Fig. 5).

encoder on the other device takes longer. Results are shown in Fig. 6. We can easily observe that the Sub-model Parallelism method can reduce the runtime of the encoder and head sub-models of the multi-modal network with multi-task heads, ultimately achieving a latency reduction of around 20% to 30% for 2 devices, and over 50% for 3 devices. For cases with same number of devices, the specific reduction ratio of this latency depends on the time allocation among the encoder, fusion, and head components, as well as the time taken by each individual encoder sub-model. And for more devices with more encoder and head sub-models, the degree of acceleration can theoretically increase by multiples. If the encoder time is between $10ms$ and $50ms$, the fusion time is between $5ms$ and $15ms$, and the head time is between $5ms$ and $35ms$, then through simulation, in the ideal scenario, the speed-up ratio achieved by our method is shown in Fig. 7. This shows that our method has relatively good capability in scaling multi-modal DNNs with multi-task heads across multiple edge devices.

V. IMPLICATIONS

Our proposed solution, Sub-model Parallelism, can be applied in many edge computing scenarios where multi-modal DNNs are widely used, such as intelligent robots and autonomous vehicles. By assigning different sub-models to different devices and fusing encoded outputs on one device during the fusion operation, our proposed solution makes latency reduction available for multiple edge devices.

To be specific, Sub-model Parallelism is a scale-out method that achieves acceleration in edge scenario by having multiple devices process the sub-models of a large and structured DNN model in parallel. For certain applications with multiple modal inputs and capable of outputting results for multiple tasks, the execution latency can be reduced due to parallelism. This method of achieving scalable latency reduction for a scale-out edge device cluster with high-speed interconnect is effective and easy to implement.

In a word, our findings offer an efficient deployment approach for multi-modal DNNs with multi-task heads running on edge application scenarios with ultra-low latency requirements, leveraging the capability of multiple edge devices through scaling out. By effectively reducing runtime latency

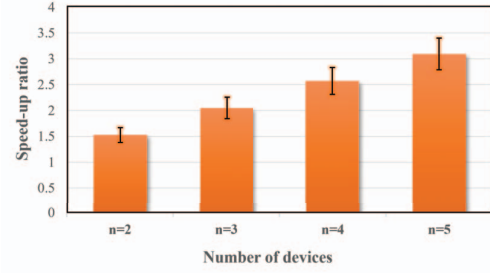


Fig. 7. Ideal speed-up ratio when scaling-out the number of devices based on a given running time range of sub-models.

at the edge, our solution contributes to the advancement of edge computing technologies and their broader adoption in real-world applications.

VI. CONTRAST

There are various solutions to deploy large inference tasks on edge devices, while our approach has its own advantages. Existing methods for facilitating the deployment of large models on edge devices can be broadly categorized into two primary approaches: *optimization for single device* and *distributed processing across multiple devices*.

A. Single Device Scenario

For a single device, model trimming technologies including quantization, compression, pruning and knowledge distillation have experienced fast development [22], [24]. These techniques allow for the reduction of computational demands for efficient deployment of DNNs. However, the achievement of reduced computational demands comes at the cost of accuracy degradation. The compromise in model prediction accuracy raises concerns, especially in applications where maintaining a high level of accuracy is paramount like autonomous driving. Our methodology of Sub-model Parallelism requires no modification on the weight parameters of a given mode and hence brings no accuracy degradation.

As for using a single device, the scalability of scale-up is limited because of power, space and cost factors. Enhancing the computational power of an individual device significantly is challenging. In such cases, scale-out would be necessary.

B. Multiple Devices Scenario

In the context of multiple devices, various parallelism approaches are developed for execution speedup.

Cloud offloading is a common strategy, leveraging the computational resources of remote servers [26]. While this approach provides virtually unlimited computing capability, it introduces additional data transfer latency, and exerts pressure on the transmission bandwidth. While this trade-off may be acceptable in some scenarios, it is not practical for real-time processing with ultra-low latency requirement.

Layer-wise parallelism entails the distribution of different layers of the network across multiple devices to enhance

computational efficiency [21], [23]. However, its implementation complexity and associated high communication costs present challenges that may limit its applicability in latency-sensitive scenarios. Layer-wise parallelism typically involves analyzing various layers of the model to discern their parallel characteristics and necessitates the transmission of information between layers across devices. For models already composed of sub-models, such as BEVFusion model discussed above, our approach directly implements parallelism at the sub-model level, obviating the need for intricate modification of the original model.

Pipeline parallelism stands out as a mature approach to greatly improve system throughput [25]. However, in latency-constrained scenarios, pipeline parallelism fails to achieve latency speedup. This limitation arises due to the sequential nature of pipeline processing, where each stage depends on the completion of the previous one, leading to potential bottlenecks and delays in meeting stringent latency requirements. In application scenarios with multiple different latency requirements, such as autonomous driving, directly employing a pipeline approach may lead to situations where tasks with higher latency requirements are blocked by tasks with lower latency requirements, thereby exerting an adverse influence on the completion of tasks. Our approach can treat the BEV algorithm as a regular routine. This method not only meets the ultra-low latency requirements of the BEVFusion network, but also accommodates other non-BEV tasks.

VII. CONCLUSION

We propose *Sub-model Parallelism*, a parallel inference method that targets the ultra-low latency requirements of multi-modal DNNs with multi-task heads on an edge device cluster with high-speed interconnection. Typical execution scenarios where this setup apply include autonomous driving and intelligent robots. Contrary to conventional parallel techniques, our parallel approach effectively reduces the total network latency without frequent communication. We expect that our work can help provide valuable experience for scalable deployment of large multi-modal DNNs with multiple task heads on multiple edge devices.

ACKNOWLEDGMENTS

We thank all the anonymous reviewers for their valuable comments and suggestions. This work is supported by the National Natural Science Foundation of China (No. 62122053). Tianhao Huang and Lingyu Sun have contributed equally. The corresponding authors are Xiaofeng Hou and Chao Li.

REFERENCES

- [1] Baltrušaitis, Tadas, Chaitanya Ahuja, and Louis-Philippe Morency. "Multimodal machine learning: A survey and taxonomy." *IEEE transactions on pattern analysis and machine intelligence* 41.2 (2018): 423-443.
- [2] Wang, Weiyao, Du Tran, and Matt Feiszli. "What makes training multi-modal classification networks hard?." *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2020.
- [3] Zhang, Chao, et al. "Multimodal intelligence: Representation learning, information fusion, and applications." *IEEE Journal of Selected Topics in Signal Processing* 14.3 (2020): 478-493.
- [4] Li C, Xue Y, Wang J, et al. Edge-oriented computing paradigms: A survey on architecture design and system management[J]. *ACM Computing Surveys (CSUR)*, 2018, 51(2): 1-34.
- [5] Zhou, Li, et al. "Adaptive parallel execution of deep neural networks on heterogeneous edge devices." *Proceedings of the 4th ACM/IEEE Symposium on Edge Computing*. 2019.
- [6] Lin, Sen, et al. "On-Demand Accelerating Deep Neural Network Inference via Edge Computing." *Edge Intelligence in the Making: Optimization, Deep Learning, and Applications*. Cham: Springer International Publishing, 2021. 151-168.
- [7] Jankowski, Mikolaj, Deniz Gündüz, and Krystian Mikolajczyk. "Joint device-edge inference over wireless links with pruning." *2020 IEEE 21st international workshop on signal processing advances in wireless communications (SPAWC)*. IEEE, 2020.
- [8] Zhou, Li, et al. "Adaptive parallel execution of deep neural networks on heterogeneous edge devices." *Proceedings of the 4th ACM/IEEE Symposium on Edge Computing*. 2019.
- [9] Hou X, Liu J, Tang X, et al. Mnexit: Enabling fast and efficient multi-modal dnn inference with adaptive network exits[C]//*European Conference on Parallel Processing*. Cham: Springer Nature Switzerland, 2023: 426-440.
- [10] Hou X, Tang P, Li C, et al. SMG: A System-Level Modality Gating Facility for Fast and Energy-Efficient Multimodal Computing[C]//*2023 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2023: 291-303.
- [11] Huang, Yu, et al. "What makes multi-modal learning better than single (provably)." *Advances in Neural Information Processing Systems* 34 (2021): 10944-10956.
- [12] Liang, Paul Pu, et al. "Multibench: Multiscale benchmarks for multi-modal representation learning." *arXiv preprint arXiv:2107.07502* (2021).
- [13] Xu C, Hou X, Liu J, et al. MMBench: Benchmarking End-to-End Multi-modal DNNs and Understanding Their Hardware-Software Implications[C]//*2023 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2023: 154-166.
- [14] Hou X, Xu C, Liu J, et al. Characterizing and understanding end-to-end multi-modal neural networks on gpus[J]. *IEEE Computer Architecture Letters*, 2022, 21(2): 125-128.
- [15] Caesar, Holger, et al. "nuscenes: A multimodal dataset for autonomous driving." *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2020.
- [16] Lee, Michelle A., et al. "Making sense of vision and touch: Self-supervised learning of multimodal representations for contact-rich tasks." *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019.
- [17] Lin, Shih-Chieh, et al. "The architectural implications of autonomous driving: Constraints and acceleration." *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems*. 2018.
- [18] Campolo, Claudia, et al. "Better platooning control toward autonomous driving: An LTE device-to-device communications strategy that meets ultralow latency requirements." *IEEE Vehicular Technology Magazine* 12.1 (2017): 30-38.
- [19] Voigtländer, Florian, et al. "5G for robotics: Ultra-low latency control of distributed robotic systems." *2017 International Symposium on Computer Science and Intelligent Controls (ISCSIC)*. IEEE, 2017.
- [20] Liu, Zhijian, et al. "BEVFusion: Multi-task multi-sensor fusion with unified bird's-eye view representation." *2023 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2023.
- [21] H. Zhou, M. Li, N. Wang, G. Min and J. Wu, "Accelerating Deep Learning Inference via Model Parallelism and Partial Computation Offloading," in *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 2, pp. 475-488, 1 Feb. 2023, doi: 10.1109/TPDS.2022.3222509.
- [22] Han, S., Mao, H. and Dally, W.J., 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. *arXiv preprint arXiv:1510.00149*.
- [23] J. Du et al., "Model Parallelism Optimization for Distributed Inference Via Decoupled CNN Structure," in *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 7, pp. 1665-1676, 1 July 2021, doi: 10.1109/TPDS.2020.3041474.
- [24] L. Deng, G. Li, S. Han, L. Shi and Y. Xie, "Model Compression and Hardware Acceleration for Neural Networks: A Comprehensive Survey," in *Proceedings of the IEEE*, vol. 108, no. 4, pp. 485-532, April 2020, doi: 10.1109/JPROC.2020.2976475.
- [25] Y. Hu et al., "PipeEdge: Pipeline Parallelism for Large-Scale Model Inference on Heterogeneous Edge Devices," *2022 25th Euromicro Con-*

- ference on Digital System Design (DSD), Maspalomas, Spain, 2022, pp. 298-307, doi: 10.1109/DSD57027.2022.00048.
- [26] B. Wang, C. Wang, W. Huang, Y. Song and X. Qin, "A Survey and Taxonomy on Task Offloading for Edge-Cloud Computing," in *IEEE Access*, vol. 8, pp. 186080-186101, 2020, doi: 10.1109/ACCESS.2020.3029649.
 - [27] Z. Chang, S. Liu, X. Xiong, Z. Cai and G. Tu, "A Survey of Recent Advances in Edge-Computing-Powered Artificial Intelligence of Things," in *IEEE Internet of Things Journal*, vol. 8, no. 18, pp. 13849-13875, 15 Sept.15, 2021, doi: 10.1109/JIOT.2021.3088875.
 - [28] P. Joshi, M. Hasanuzzaman, C. Thapa, H. Afli and T. Scully, "Enabling All In-Edge Deep Learning: A Literature Review," in *IEEE Access*, vol. 11, pp. 3431-3460, 2023, doi: 10.1109/ACCESS.2023.3234761.
 - [29] Jensen, Morten Bornø, et al. "Vision for looking at traffic lights: Issues, survey, and perspectives." *IEEE transactions on intelligent transportation systems* 17.7 (2016): 1800-1815.
 - [30] Bingol, Mustafa Can, and Omur Aydogmus. "Performing predefined tasks using the human-robot interaction on speech recognition for an industrial robot." *Engineering Applications of Artificial Intelligence* 95 (2020): 103903.
 - [31] Yu, Chenglin, and Hailong Pei. "Face recognition framework based on effective computing and adversarial neural network and its implementation in machine vision for social robots." *Computers & Electrical Engineering* 92 (2021): 107128.
 - [32] linux rdma, "perftest," <https://github.com/linux-rdma/perftest>, [Online; accessed 19-Decemember-2023].
 - [33] NVIDIA AI IOT, "Lidar_AI_Solution," https://github.com/NVIDIA-AI-IOT/Lidar_AI_Solution, [Online; accessed 19-Decemember-2023].